

AI Assisted Coding

ASSIGNMENT 16.3

Name: K. Rupaas shyni

HT No: 2303A52417

Batch: 32

Task 1 – Hospital Management Database

Task:

Create schema and queries for a **Hospital Management System**.

Instructions:

- Tables: Doctors, Patients, Appointments.
- Use AI to define constraints (unique IDs, valid dates).
- Generate queries:
 - List all appointments for a specific doctor.
 - Retrieve patient history by patient ID.
 - Count total patients treated by each doctor.

Expected Output:

- Normalized schema and SQL queries with joins.

Prompt:

Create SQLite schema for Doctors, Patients, Appointments with PK, FK, constraints.

Write queries:

- appointments by doctor
- patient history by id
- patient count per doctor

Use JOINS.

Code:

```
--Drop tables if they exist
DROP TABLE IF EXISTS Appointments;
DROP TABLE IF EXISTS Doctors;
DROP TABLE IF EXISTS Patients;

-- Create Doctors table
CREATE TABLE Doctors (
    doctor_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    specialty TEXT NOT NULL
);

-- Create Patients table
CREATE TABLE Patients (
    patient_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    date_of_birth DATE NOT NULL
);

-- Create Appointments table
CREATE TABLE Appointments (
    appointment_id INTEGER PRIMARY KEY,
    doctor_id INTEGER NOT NULL,
    patient_id INTEGER NOT NULL,
    appointment_date DATE NOT NULL,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);

--- Insert sample data into Doctors
INSERT INTO Doctors (name, specialty) VALUES ('Dr. Smith',
'Cardiology');
INSERT INTO Doctors (name, specialty) VALUES ('Dr. Johnson',
'Neurology');
INSERT INTO Doctors (name, specialty) VALUES ('Dr. Lee', 'Pediatrics');
--- Insert sample data into Patients
INSERT INTO Patients (name, date_of_birth) VALUES ('Alice Brown',
'1980-05-15');
INSERT INTO Patients (name, date_of_birth) VALUES ('Bob Green', '1990-
08-22');
INSERT INTO Patients (name, date_of_birth) VALUES ('Charlie White',
'2000-12-01');
-- Insert sample data into Appointments
```

```
INSERT INTO Appointments (doctor_id, patient_id, appointment_date)
VALUES (1, 1, '2023-10-15');
INSERT INTO Appointments (doctor_id, patient_id, appointment_date)
VALUES (1, 2, '2023-10-16');
INSERT INTO Appointments (doctor_id, patient_id, appointment_date)
VALUES (2, 1, '2023-10-17');

-- Query: Appointments by doctor
SELECT d.name AS doctor_name, a.appointment_date, p.name AS
patient_name
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
JOIN Patients p ON a.patient_id = p.patient_id
ORDER BY d.name, a.appointment_date;

-- Query: Patient history by id
SELECT p.name AS patient_name, p.date_of_birth, d.name AS doctor_name,
a.appointment_date
FROM Patients p
JOIN Appointments a ON p.patient_id = a.patient_id
JOIN Doctors d ON a.doctor_id = d.doctor_id
WHERE p.patient_id = 1
ORDER BY a.appointment_date;

-- Query: Patient count per doctor
SELECT d.name AS doctor_name, COUNT(a.patient_id) AS patient_count
FROM Doctors d
LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id, d.name;
```

Output:

SQLite

SQL ▾ < 1 / 1 > 1 - 3 of 3

```
SELECT d.name AS doctor_name, a.appointment_date,
p.name AS patient_name
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
JOIN Patients p ON a.patient_id = p.patient_id
ORDER BY d.name, a.appointment_date;
```

doctor_name	appointment_date	patient_name
Dr. Johnson	2023-10-17	Alice Brown
Dr. Smith	2023-10-15	Alice Brown
Dr. Smith	2023-10-16	Bob Green

SQL ▾ < 1 / 1 > 1 - 2 of 2

patient_name	date_of_birth	doctor_name	appointment
Alice Brown	1980-05-15	Dr. Smith	2023-10-15
Alice Brown	1980-05-15	Dr. Johnson	2023-10-17

SQL ▾ < 1 / 1 > 1 - 3 of 3

doctor_name	patient_count
Dr. Smith	2
Dr. Johnson	1
Dr. Lee	0

Explanation:

- Doctors, Patients, and Appointments tables are created with primary and foreign keys to maintain relationships.
- Appointments table connects doctors and patients.
- JOIN is used to combine data from multiple tables.
- Queries fetch doctor appointments, patient history, and total patients per doctor.

Task 2 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an **E-commerce platform**.

Requirements:

- Tables: Users, Products, Orders, OrderDetails.
- Generate queries to:
- Retrieve all orders by a user.
- Find the most purchased product.
- Calculate total revenue in a given month.
- AI should also suggest **normalization improvements** and **query optimization**.

Expected Output:

- Complete schema with relationships + SQL queries for analytics.

Prompt:

Create SQLite schema for Users, Products, Orders, OrderDetails with PK, FK.

Write queries:

- orders by user
- most purchased product
- monthly revenue

Suggest indexing.

Code:

```
--Drop tables if they exist
DROP TABLE IF EXISTS OrderDetails;
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Users;

-- Create tables
CREATE TABLE Users (
    UserID INTEGER PRIMARY KEY,
    UserName TEXT NOT NULL,
    Email TEXT NOT NULL UNIQUE
);
CREATE TABLE Products (
    ProductID INTEGER PRIMARY KEY,
    ProductName TEXT NOT NULL,
    Price REAL NOT NULL
);
CREATE TABLE Orders (
    OrderID INTEGER PRIMARY KEY,
    UserID INTEGER,
    OrderDate DATE,
    FOREIGN KEY (UserID) REFERENCES Users(UserID)
);
CREATE TABLE OrderDetails (
    OrderDetailID INTEGER PRIMARY KEY,
    OrderID INTEGER,
    ProductID INTEGER,
    Quantity INTEGER,
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);

-- Insert sample data
INSERT INTO Users (UserName, Email) VALUES ('Alice',
'alice@example.com'), ('Bob', 'bob@example.com');
INSERT INTO Products (ProductName, Price) VALUES ('Laptop', 1000.00),
('Mouse', 25.00);
INSERT INTO Orders (UserID, OrderDate) VALUES (1, '2023-01-15'), (2,
'2023-01-16');
INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES (1, 1,
1), (1, 2, 2), (2, 2, 1);
```

```

-- Queries
-- Orders by user
SELECT u.UserName, o.OrderID, o.OrderDate
FROM Users u
JOIN Orders o ON u.UserID = o.UserID
WHERE u.UserID = 1; -- Replace with desired UserID
-- Most purchased product
SELECT p.ProductName, SUM(od.Quantity) AS TotalQuantity
FROM Products p
JOIN OrderDetails od ON p.ProductID = od.ProductID
GROUP BY p.ProductID
ORDER BY TotalQuantity DESC
LIMIT 1;
-- Monthly revenue
SELECT strftime('%Y-%m', o.OrderDate) AS Month, SUM(p.Price *
od.Quantity) AS Revenue
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY Month
ORDER BY Month;
-- Indexing suggestions
CREATE INDEX idx_orders_user_id ON Orders(UserID);
CREATE INDEX idx_order_details_order_id ON OrderDetails(OrderID);
CREATE INDEX idx_order_details_product_id ON OrderDetails(ProductID);
CREATE INDEX idx_products_price ON Products(Price);
-- Validate indexes
PRAGMA index_list('Orders');
PRAGMA index_list('OrderDetails');
PRAGMA index_list('Products');
PRAGMA index_info('idx_orders_user_id');
PRAGMA index_info('idx_order_details_order_id');
PRAGMA index_info('idx_order_details_product_id');
PRAGMA index_info('idx_products_price');-

```


Output:

SQL ▾	<	1	/ 1	>	1 - 1 of 1	📊	↔	{ }	👁
UserName	OrderID	OrderDate							
Alice	1	2023-01-15							

SQL ▾	<	1	/ 1	>	1 - 1 of 1	📊	↔	{ }	👁
ProductName	TotalQuantity								
Mouse	3								

SQL ▾	<	1	/ 1	>	1 - 1 of 1	📊	↔	{ }	👁
Month	Revenue								
2023-01	1075.0								

Explanation:

- Users, Products, Orders, and OrderDetails tables are created and normalized.
- OrderDetails removes redundancy by storing product quantities separately.
- JOIN and aggregate functions are used for analytics queries.
- Indexes are added to improve query performance.

Index Validation:

SQLite

SQL ▾ < 1 / 1 > 1 - 1 of 1

seq	name	unique	origin	partial
0	idx_orders_user_id	0	c	0

SQL ▾ < 1 / 1 > 1 - 2 of 2

seq	name	unique	origin	partial
0	idx_order_details_product_id	0	c	0
1	idx_order_details_order_id	0	c	0

SQL ▾ < 1 / 1 > 1 - 1 of 1

seq	name	unique	origin	partial
0	idx_products_price	0	c	0

SQL ▾ < 1 / 1 > 1 - 1 of 1

seqno	cid	name
0	1	UserID

SQL ▾ < 1 / 1 > 1 - 1 of 1

seqno	cid	name
0	1	OrderID

SQL ▾ < 1 / 1 > 1 - 1 of 1

seqno	cid	name
0	2	ProductID

SQL ▾ < 1 / 1 > 1 - 1 of 1

seqno	cid	name
0	2	Price

Task 3 – Library Database

Task:

Design schema for a **Library Management System**.

Instructions:

- Tables: Books, Members, Loans.
- Use AI to suggest indexing strategy for faster queries.
- Generate queries:
- Retrieve all books currently issued.
- Find overdue books (loan date > 30 days).
- Count the number of books loaned by each member.

Expected Output:

- Schema + SQL queries demonstrating joins and conditions.

Prompt:

Create SQLite schema for Books, Members, Loans with PK, FK.

Write queries:

- currently issued books
- overdue books (>30 days)
- books count per member

Suggest indexes.

Code:

```
-- Create tables
CREATE TABLE Books (
    BookID INTEGER PRIMARY KEY,
    Title TEXT NOT NULL,
    Author TEXT NOT NULL,
    PublishedYear INTEGER
);

CREATE TABLE Members (
    MemberID INTEGER PRIMARY KEY,
    Name TEXT NOT NULL,
    JoinDate DATE
);

CREATE TABLE Loans (
    LoanID INTEGER PRIMARY KEY,
    BookID INTEGER,
    MemberID INTEGER,
    LoanDate DATE,
    ReturnDate DATE,
    FOREIGN KEY (BookID) REFERENCES Books(BookID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
);

-- Insert sample data
INSERT INTO Books (Title, Author, PublishedYear) VALUES
('The Great Gatsby', 'F. Scott Fitzgerald', 1925),
('To Kill a Mockingbird', 'Harper Lee', 1960),
('1984', 'George Orwell', 1949);
INSERT INTO Members (Name, JoinDate) VALUES
('Alice Johnson', '2020-01-15'),
('Bob Smith', '2021-06-10'),
('Charlie Brown', '2019-11-05');
INSERT INTO Loans (BookID, MemberID, LoanDate, ReturnDate) VALUES
(1, 1, '2023-01-10', NULL),
(2, 2, '2023-02-20', '2023-03-01'),
(3, 3, '2023-01-05', NULL);

-- Query: currently issued books
SELECT b.Title, m.Name, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL;
```

```
-- Query: overdue books (>30 days)
SELECT b.Title, m.Name, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL AND julianday('now') - julianday(l.LoanDate)
> 30;

-- Query: books count per member
SELECT m.Name, COUNT(l.LoanID) AS BooksCount
FROM Members m
LEFT JOIN Loans l ON m.MemberID = l.MemberID
GROUP BY m.MemberID;

-- Suggested indexes
CREATE INDEX idx_loans_bookid ON Loans(BookID);
CREATE INDEX idx_loans_memberid ON Loans(MemberID);
CREATE INDEX idx_loans_loandate ON Loans(LoanDate);
CREATE INDEX idx_loans_returndate ON Loans(ReturnDate);
```

Output:

The screenshot shows a SQLite database viewer with three query results. Each result is displayed in a table with a header row and data rows. The first two results have three columns: Title, Name, and LoanDate. The third result has two columns: Name and BooksCount.

Title	Name	LoanDate
The Great Gatsby	Alice Johnson	2023-01-10
1984	Charlie Brown	2023-01-05

Title	Name	LoanDate
The Great Gatsby	Alice Johnson	2023-01-10
1984	Charlie Brown	2023-01-05

Name	BooksCount
Alice Johnson	1
Bob Smith	1
Charlie Brown	1

Explanation:

- Books, Members, and Loans tables are linked using foreign keys.
- Loans table tracks issued and returned books.
- Queries use JOIN and conditions to find issued and overdue books.
- Indexes help speed up search operations.

Task 4: Optimize Queries

Instructions:

- Use AI tools to analyze query efficiency.
- Optimize inefficient queries using joins, indexes, or reducing subqueries.
- Test optimized queries for correctness.

Starter Code Example:

-- Original query

```
SELECT * FROM Books WHERE author_id IN (SELECT author_id  
FROM Authors WHERE country='UK');
```

-- Optimized query

```
SELECT b.*
```

```
FROM Books b
```

```
JOIN Authors a ON b.author_id = a.author_id
```

```
WHERE a.country = 'UK';
```

Expected Output:

- Both queries return the same result: all books written by UK authors.
- Optimized query performs faster for larger datasets.

Prompt:

Optimize this query using JOIN:

```
SELECT * FROM Books WHERE author_id IN  
(SELECT author_id FROM Authors WHERE country='UK');
```

Code:

```
-- Create tables  
CREATE TABLE Authors (  
    AuthorID INTEGER PRIMARY KEY,  
    Name TEXT NOT NULL,  
    Country TEXT NOT NULL  
);  
CREATE TABLE Books (  
    BookID INTEGER PRIMARY KEY,  
    Title TEXT NOT NULL,  
    AuthorID INTEGER,  
    PublishedYear INTEGER,  
    FOREIGN KEY (AuthorID) REFERENCES Authors (AuthorID)  
);  
-- Insert sample data  
INSERT INTO Authors (Name, Country) VALUES  
('J.K. Rowling', 'UK'),  
('George Orwell', 'UK'),  
('F. Scott Fitzgerald', 'USA');  
INSERT INTO Books (Title, AuthorID, PublishedYear) VALUES  
('Harry Potter and the Sorcerer's Stone', 1, 1997),  
('1984', 2, 1949),  
('The Great Gatsby', 3, 1925);  
-- Original query  
SELECT * FROM Books WHERE AuthorID IN  
(SELECT AuthorID FROM Authors WHERE Country='UK');  
-- Optimized query using JOIN  
SELECT b.*  
FROM Books b  
JOIN Authors a ON b.AuthorID = a.AuthorID  
WHERE a.Country = 'UK';
```


Output:

The screenshot shows a SQLite IDE window with two SQL queries and their results. The first query is a subquery, and the second is a JOIN query. Both return the same two rows of book data.

Query 1 (Subquery):

```
SQL < 1 / 1 > 1 - 0 of 0
```

Query 2 (Subquery):

```
SQL < 1 / 1 > 1 - 0 of 0
```

Query 3 (Subquery):

```
SQL < 1 / 1 > 1 - 0 of 0
```

Query 4 (Subquery):

```
SQL < 1 / 1 > 1 - 0 of 0
```

Query 5 (Subquery):

```
SQL < 1 / 1 > 1 - 0 of 0
```

Query 6 (Subquery):

```
SQL < 1 / 1 > 1 - 2 of 2
```

BookID	Title	AuthorID	PublishedYear
1	Harry Potter and the Sorcerer's Stone	1	1997
2	1984	2	1949

Query 7 (JOIN):

```
SQL < 1 / 1 > 1 - 2 of 2
```

BookID	Title	AuthorID	PublishedYear
1	Harry Potter and the Sorcerer's Stone	1	1997
2	1984	2	1949

Explanation:

- Original query uses subquery which is slower for large data.
- Optimized query uses JOIN to directly match tables.
- JOIN improves performance and reduces execution time.
- Both queries give same result but optimized one is faster.

Task 5: University Course Registration

Scenario:

SR University needs a **course registration system** for students enrolling in different courses under faculty supervision.

- Use Copilot to design schema tables: Students, Courses, Faculty, and Registrations.
- Define relationships:
- Each student can register for multiple courses.
- Each course is taught by a faculty member.
- Write SQL queries to:
- List all students enrolled in a specific course.
- Find faculty members teaching more than 2 courses.
- Retrieve courses with the highest number of registrations.

Prompt:

Create SQLite schema for Students, Courses, Faculty, Registrations with relationships.

Write queries:

- students in a course
- faculty teaching >2 courses
- most registered courses

Use JOINS.

Code:

```
-- Drop tables if they exist
DROP TABLE IF EXISTS Registrations;
DROP TABLE IF EXISTS Students;
DROP TABLE IF EXISTS Courses;
DROP TABLE IF EXISTS Faculty;

-- Create tables
CREATE TABLE Faculty (
    faculty_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL
```

```

);
CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY,
    course_name TEXT NOT NULL,
    faculty_id INTEGER,
    FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id)
);
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL
);
CREATE TABLE Registrations (
    registration_id INTEGER PRIMARY KEY,
    student_id INTEGER,
    course_id INTEGER,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

-- Sample data insertion
INSERT INTO Faculty (name) VALUES ('Dr. Smith'), ('Dr. Johnson'), ('Dr. Lee');
INSERT INTO Courses (course_name, faculty_id) VALUES ('Math 101', 1), ('Math 103', 1), ('History 201', 2), ('Science 301', 3), ('Math 102', 1), ('History 202', 2), ('Science 302', 3);
INSERT INTO Students (name) VALUES ('Alice'), ('Bob'), ('Charlie');
INSERT INTO Registrations (student_id, course_id) VALUES (1, 1), (1, 2), (2, 1), (3, 3);
INSERT INTO Registrations (student_id, course_id) VALUES (2, 3), (3, 1), (3, 2);

-- Query: students in a course
SELECT s.name AS student_name, c.course_name
FROM Students s
JOIN Registrations r ON s.student_id = r.student_id
JOIN Courses c ON r.course_id = c.course_id
WHERE c.course_name = 'Math 101';

-- Query: faculty teaching >2 courses
SELECT f.name AS faculty_name, COUNT(c.course_id) AS course_count
FROM Faculty f
JOIN Courses c ON f.faculty_id = c.faculty_id
GROUP BY f.faculty_id, f.name

```

```
HAVING COUNT(c.course_id) > 2;
```

```
-- Query: most registered courses
```

```
SELECT c.course_name, COUNT(r.registration_id) AS registration_count  
FROM Courses c  
JOIN Registrations r ON c.course_id = r.course_id  
GROUP BY c.course_id, c.course_name  
ORDER BY registration_count DESC  
LIMIT 1;
```

Output:

The screenshot shows a SQLite IDE interface with three SQL queries and their results. Each query is displayed in a separate panel with a toolbar containing icons for SQL, navigation, and execution.

Query 1: SQL ▾ < 1 / 1 > 1 - 0 of 0

student_name	course_name
Alice	Math 101
Bob	Math 101
Charlie	Math 101

Query 2: SQL ▾ < 1 / 1 > 1 - 1 of 1

faculty_name	course_count
Dr. Smith	3

Query 3: SQL ▾ < 1 / 1 > 1 - 1 of 1

course_name	registration_count
Math 101	3

Explanation:

- Students, Courses, Faculty, and Registrations tables are created with relationships.
- The registrations table handles many-to-many relationships between students and courses.
- JOIN is used to retrieve combined data.
- Aggregate functions help find faculty load and popular courses.